mHD

Minimal Hotkey Daemon for Windows

One native tray process for hotkeys, remaps, monitor control, audio control, notes, timers, and small desktop tools.

mHD is a small Windows utility daemon for people who are tired of installing, launching, and keeping several separate tools around just to get a practical desktop workflow.

The intent is to cover the everyday power-user actions that often end up split across products such as Windows PowerToys, monitor utilities, audio mixers, key remappers, launcher scripts, small note tools, timer apps, and window helpers. mHD puts those jobs behind one native tray process, one config file, and one hotkey system.

It is not a replacement for every large productivity suite. The point is narrower: keep the useful parts close, fast, local, and lightweight.

mHD ships as a single `mhd.exe`. It does not install a driver, does not require a service, does not need WebView2, and does not bring a large UI framework just to draw a few utility windows.

The product is built around a simple idea:

- one background daemon;
- one tray icon;
- one TOML config;
- native Win32 overlays;
- hotkeys for the tools you actually use;
- no separate helper apps for every small task.

At A Glance

Area	What mHD does
Input	Keyboard remaps, mouse bindings, scheme switching
Automation	PowerShell commands, program launch, hotkey-driven actions
Display	DDC/CI brightness, VCP control, monitor panel, brightness OSD
Audio	Master volume, per-app mixer, media key actions
Desktop tools	Quick Note, Quick Draw, Pomodoro, power panel, CPU plan panel
LLM Proxy	Local proxy for Claude Code — intercepts API calls, remaps models
Window/process control	Always-on-top, suspend-on-blur, throttle-on-blur
Settings	Native config editor, theme selection, autostart, shortcut editing

Positioning

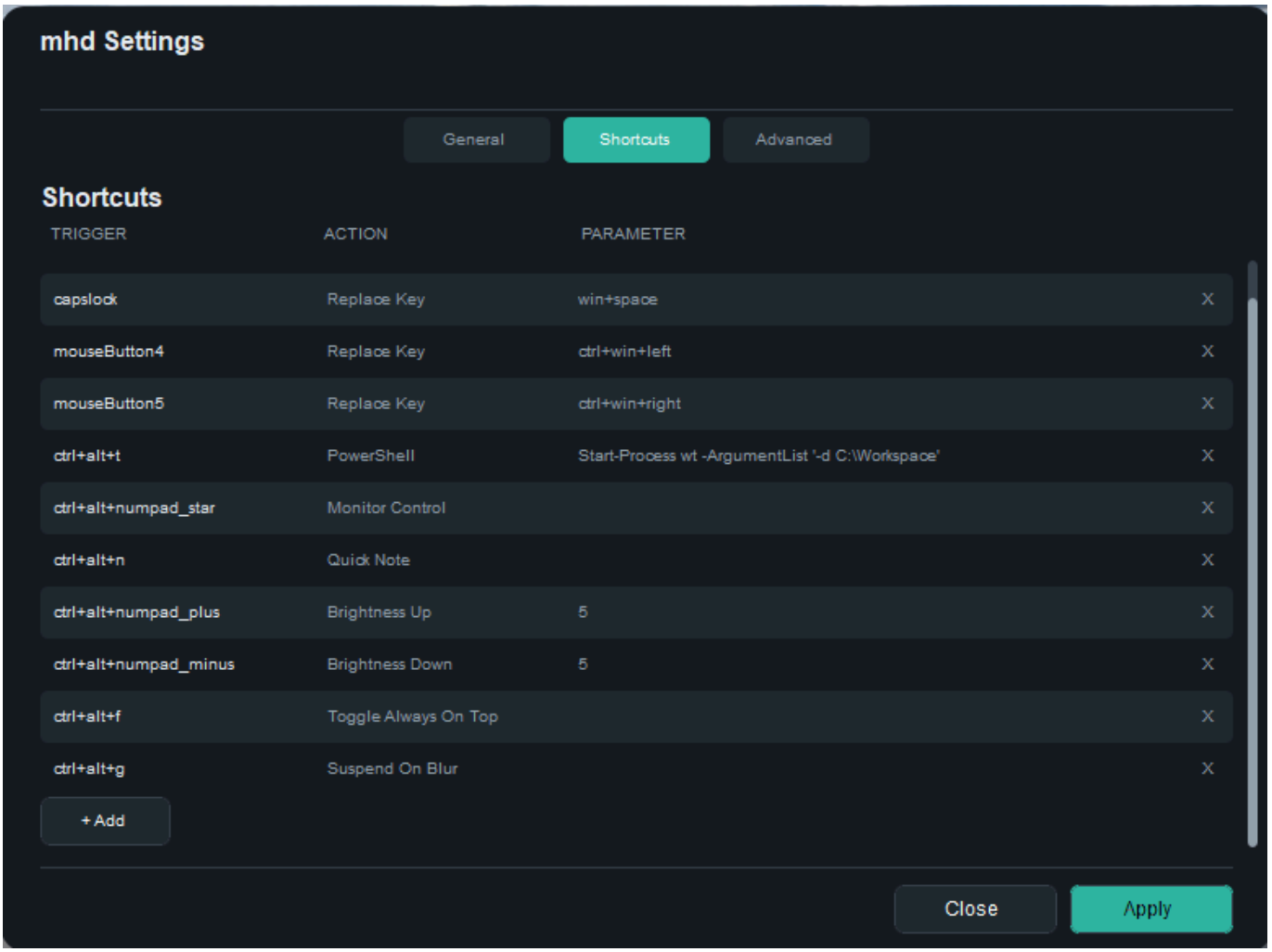
mHD is a compact desktop control layer for Windows.

It is useful when you want one tool to handle:

- keyboard and mouse remapping;
- shortcut-driven automation;
- monitor brightness and DDC/CI control;
- per-app volume control;
- media keys;
- quick notes;
- quick drawing;
- Pomodoro timing;
- power plan switching;
- window always-on-top toggling;
- process suspend/throttle behaviour;
- small native control panels available from hotkeys or tray.

The target user is someone who knows what they want their desktop to do and would rather configure a small daemon than keep a stack of unrelated utilities running.

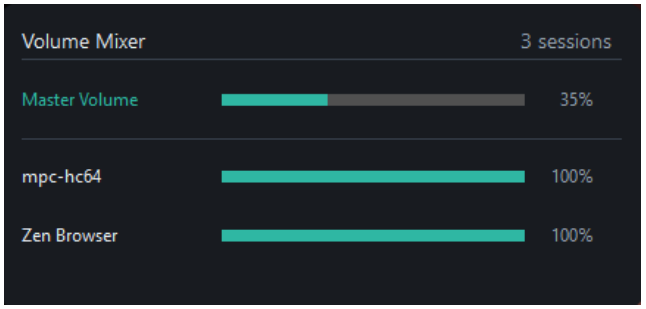
Screenshots



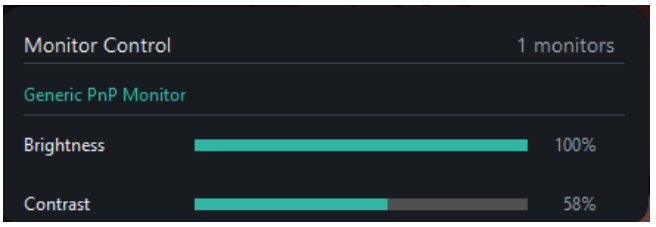
Config editor with shortcut bindings and action selection.

Native utility panels

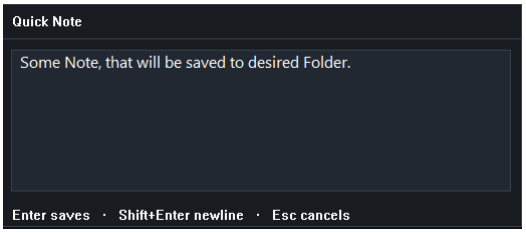
Volume Mixer



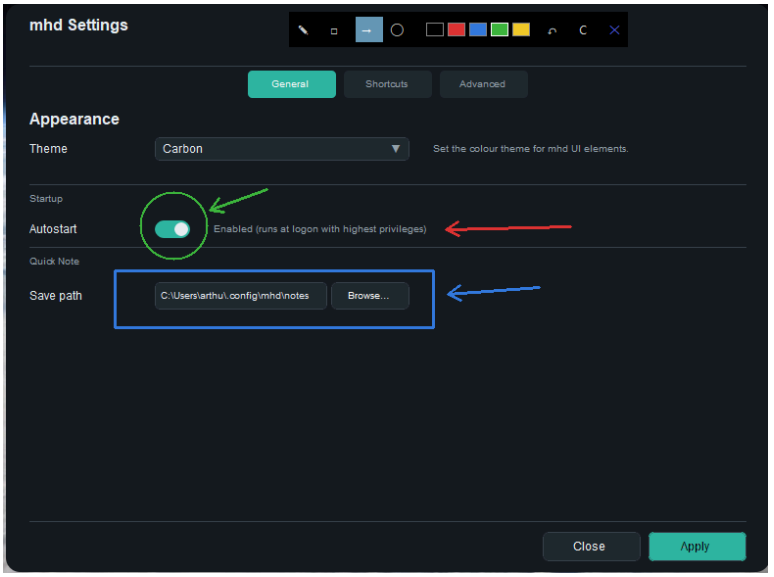
Monitor Control



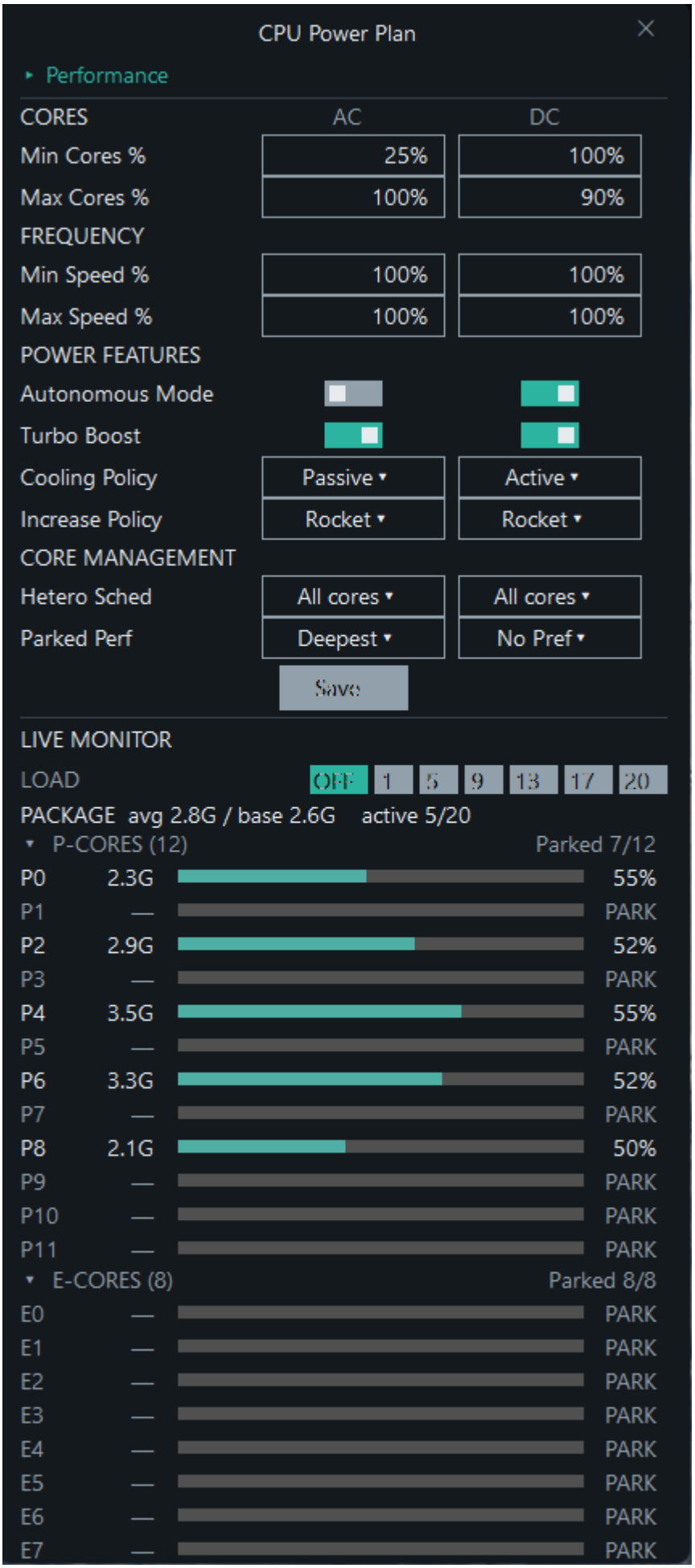
Quick Note



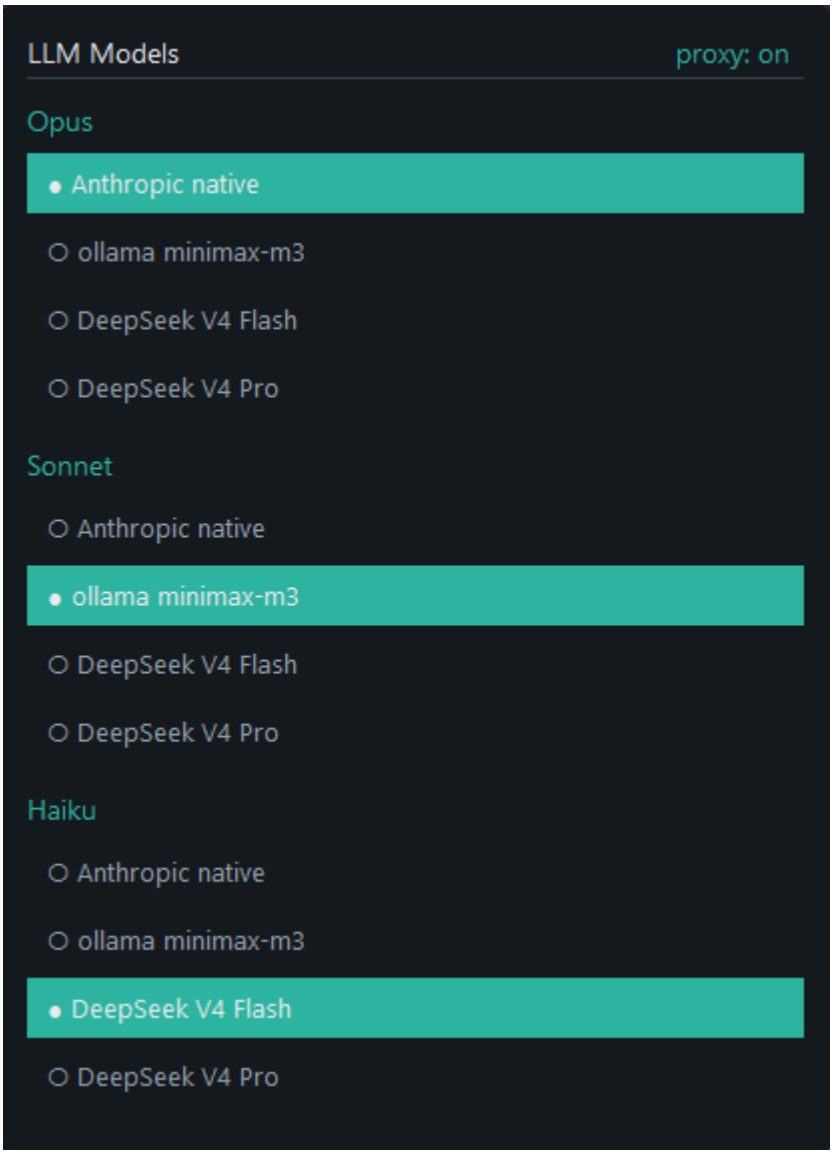
Quick Draw



CPU power plan panel



LLM Model Selector



Quick-model-switcher for the built-in LLM proxy. Cycles through configured gateway models on the fly without restarting the proxy or Claude Code.

Runtime Model

`mhd.exe` runs as one process with several internal threads:

```
mhd.exe
├── Tray thread
│   ├── system tray icon and menu
│   ├── About dialog
│   ├── config editor
│   └── quick access to utility overlays
├── Hook thread
│   ├── WH_KEYBOARD_LL and WH_MOUSE_LL hooks
│   └── trigger capture and suppression
├── Worker thread
│   ├── action dispatch
│   ├── PowerShell / program launch
│   ├── DDC/CI monitor control
```

```
│   └── power plan switching
├── OSD thread
│   └── native brightness overlay
└── Overlay threads
    ├── volume mixer
    ├── monitor panel
    ├── power panel
    ├── quick draw
    ├── quick note
    ├── pomodoro timer
    ├── CPU power panel
    └── other native utility windows
```

The design goal is to keep the daemon responsive and the UI native. The hooks do their work without dragging in heavyweight UI frameworks or polling loops.

Product Features

Hotkey and mouse bindings

mHD listens for low-level keyboard and mouse events and maps them to actions. A binding can:

- replace a key or shortcut with another key sequence;
- launch a program;
- run a PowerShell command;
- open a utility overlay;
- switch binding schemes;
- control audio, brightness, or power state;
- toggle process behaviour such as suspend/throttle/topmost.

Bindings are configured in TOML. The action parser supports both simple one-field actions and structured actions with parameters.

Key remapping

`replace_key` suppresses the original input and emits a replacement key combo through `SendInput`. This is the core remapping feature for key layout changes and ergonomic remaps.

Examples:

- `capslock -> alt+shift`
- side mouse button -> `ctrl`
- a dead key -> a custom shortcut

Mouse bindings

Mouse buttons can be bound the same way as keyboard triggers. The codepath handles button down/up suppression so a remap behaves like a real input replacement rather than a duplicate event.

DDC/CI monitor control

mHD can talk to monitors through DDC/CI and supports:

- absolute brightness changes;
- relative brightness changes;
- arbitrary VCP codes;
- a monitor panel overlay for direct adjustment.

This is useful for external displays that expose brightness, input select, or other VCP controls over the I2C/DDC path.

Brightness OSD

When brightness changes, mHD can show a native OSD. It is built as a layered Win32 window, not as a framework widget. The OSD is meant to be minimal, readable, and quick to dismiss.

Volume mixer

The built-in mixer shows:

- master output volume;
- individual per-application audio sessions;
- per-row mute and volume adjustment;
- mouse wheel volume changes while hovering a row.

It is designed as an interactive native window for day-to-day control without opening the full Windows sound UI.

Power controls

mHD includes actions for:

- switch to sleep / shutdown / wake-oriented power actions;
- switch active Windows power plans;
- edit processor power settings from the CPU power panel;
- inspect live per-core CPU load and core topology;
- toggle process suspension when a window loses focus;
- toggle aggressive throttling when a process loses focus;
- toggle always-on-top for the focused window.

These actions are intended for people who want a small, direct utility layer rather than a large automation suite.

Utility overlays

The included overlays are:

- About dialog
- Monitor panel
- Power control panel
- Quick Draw
- Quick Note

- Pomodoro timer
- CPU power panel

Each one is a native window with a narrow task and a minimal control surface.

Quick Draw is a transparent drawing layer with pencil, rectangle, circle, and arrow tools, a small colour picker, undo, clear, and close controls. Drawings remain in memory while the overlay is hidden.

Quick Note is a hotkey popup for short Markdown notes. Enter saves, Shift+Enter inserts a new line, Escape cancels, and a second hotkey press closes the popup without saving. Notes are written by day into the configured notes directory.

The Pomodoro tool keeps its timer state in a background thread, so the overlay can be opened and closed without losing the current timer. It supports task text, start/pause/stop, five-minute extension, break timing, and completion feedback.

The CPU power panel can switch Windows power plans, edit processor parking and frequency-related power settings, show live per-core load, expose P/E core topology when Windows reports it, and run a small local stress load for testing plan behaviour.

LLM Proxy

mHD includes a local LLM proxy for Claude Code. Its main job is runtime model switching: start Claude Code once through the proxy, then change the model route from the tray or a hotkey without restarting Claude Code, mHD, or the current conversation.

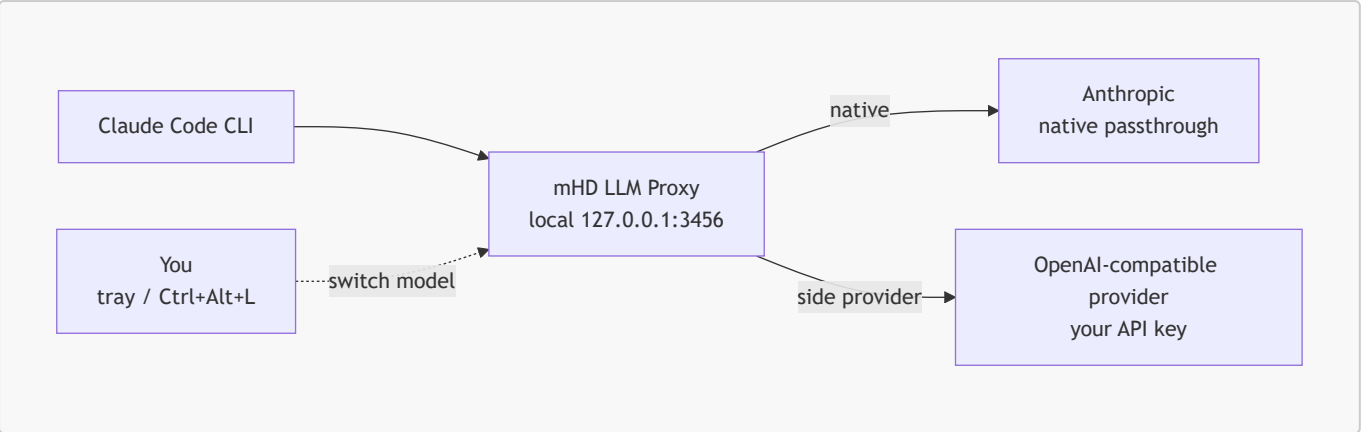
Typical flow:

1. Run mHD.
2. Start Claude Code through `claude-mhd.bat`, for example `C:\Workspace\Active\mhd\claude-mhd.bat`. The wrapper points `ANTHROPIC_BASE_URL` at the local proxy on `127.0.0.1:3456`.
3. Open **System tray -> mHD -> right click -> Settings -> LLM Proxy**.
4. Add an OpenAI-compatible provider, API key, and models.
5. Assign a shortcut for `show_llm_models` on the **Shortcuts** page. For example, `Ctrl+Alt+L`.
6. Press the shortcut while Claude Code is running and choose the target model for `opus`, `sonnet`, or `haiku`.

The selected route applies to the next Claude Code request. In-flight streams continue on the model they started with, so switching is safe during an active session.

Routing targets can be:

- `native` - pass through to Anthropic, forwarding Claude Code's OAuth token.
- any configured OpenAI-compatible model - use your provider API key for the next request.



See llm-proxy/README.md for setup details, provider configuration, runtime switching, trace view, and the proxy configuration files.

Config editor and tray

The tray provides the operational entry point:

- reload config;
- edit config;
- open utility windows;
- inspect and toggle supported features;
- switch themes;
- enable or disable autostart;
- choose the Quick Note directory;
- open config, log, and crash-log locations;
- switch LLM proxy models;
- reset shortcuts or all settings;
- quit cleanly.

The config editor is a native Win32 interface with tabs for appearance, shortcuts, and advanced maintenance. Shortcut editing uses the same action registry as the TOML parser, so new action names and parameter types stay consistent between hand-written config and the UI.

Diagnostics

Normal builds write panic details to:

```
%USERPROFILE%\.config\mhd\crash.log
```

Developer builds can also be compiled with `debug-dump` to write minidumps and an additional debug log under `%TEMP%`:

```
cargo build --release --features debug-dump
```

Build

Default build

```
cargo build --release
```

This produces the normal public build. Optional developer features are disabled by default.

Release package

The public release archive is portable and contains the normal build only:

```
.\scripts\package-release.ps1 -Version 0.2.0
```

The script creates:

```
dist\mhd-v0.2.0-windows-x64.zip
dist\mhd-v0.2.0-windows-x64.zip.sha256
```

Attach both files to the matching GitHub Release tag, for example `v0.2.0`.

Run

```
.\mhd.exe              # tray + daemon
.\mhd.exe --daemon     # headless, no tray
.\mhd.exe --quiet      # suppress startup messages
.\mhd.exe --debug-quicknote  # print QuickNote lifecycle/debug events
```

Configuration

On first run, mHD creates:

```
%USERPROFILE%\.config\mhd\config.toml
```

You can override the config path with:

```
$env:MHD_CONFIG = "C:\path\to\config.toml"
```

The main config file is TOML. It can define:

- startup binding scheme;
- active theme;
- volume step;
- autostart;
- quick note settings;
- power plan order;
- optional developer-only blackbox settings when compiled with that feature;
- bindings.

LLM proxy settings are managed through the native settings UI and stored under:

```
%USERPROFILE%\.config\mhd\llm-proxy\
```

Actions

The action list below matches the parser in `mhd-daemon/src/core/action.rs`.

Input

Action	Fields	Behaviour
`replace_key`	`keys`	Suppress the trigger and emit a replacement key combo via `SendInput`.
`run_program`	`path`	Launch a program or file path directly.

Automation

Action	Fields	Behaviour
`run_ps`	`command`	Run a PowerShell command.
`switch_scheme`	`target_scheme`	Change the active binding scheme at runtime.

Display

Action	Fields	Behaviour
`set_brightness`	`value`	Set brightness absolutely or relatively, depending on prefix.
`brightness_up`	`value`	Increase brightness by the configured step.
`brightness_down`	`value`	Decrease brightness by the configured step.
`vcp`	`code`, `value`	Set or adjust an arbitrary VCP control code.
`show_monitor_panel`	-	Open the monitor control overlay.

Audio / Media

Action	Fields	Behaviour
`show_volume_mixer`	-	Open the interactive mixer window.
`media_volume_up`	-	Send one volume-up media key step.
`media_volume_down`	-	Send one volume-down media key step.
`media_mute`	-	Toggle system mute.
`media_play_pause`	-	Play or pause the active media session.
`media_stop`	-	Stop playback.
`media_last_track`	-	Go to the previous track.
`media_next_track`	-	Go to the next track.

System

Action	Fields	Behaviour
`toggle_topmost`	-	Toggle always-on-top for the focused window.
`toggle_suspend_on_blur`	-	Suspend the focused process when it loses focus.
`toggle_throttle_on_blur`	-	Apply Windows power throttling and one-CPU affinity when a process loses focus.
`power_actions`	-	Open the power actions panel.
`switch_power_plan`	`target`	Switch to a named power plan or `next`.
`show_cpu_panel`	-	Open the CPU power plan panel.
`quit`	-	Shut mHD down cleanly.

Tools

Action	Fields	Behaviour
`quick_draw`	-	Open the quick drawing overlay.
`quick_note`	-	Open the quick note overlay.
`pomodoro`	-	Open the Pomodoro timer overlay.

LLM Proxy

Action	Fields	Behaviour
`show_llm_models`	-	Open the model selector overlay to switch proxy routing on the fly.
`toggle_llm_proxy`	-	Enable or disable the LLM proxy server at runtime.

Example Bindings

The main use case for `replace_key` is taking a shortcut that is annoying, hard to reach, or difficult to change in Windows or in another application, and mapping it to something easier. mHD can turn one key or mouse button into a full key combination, so you do not have to accept the defaults chosen by Windows or by the app.

```
# Quit mHD.
[[binding]]
trigger = "ctrl+alt+f12"
action = "quit"

# CapsLock -> Alt+Shift.
# Useful when Alt+Shift is your Windows keyboard layout switch:
# one key changes language/layout instead of a two-key chord.
[[binding]]
trigger = "capslock"
action = "replace_key"
keys = "alt+shift"

# Mouse side buttons -> virtual desktop navigation.
# Windows uses Ctrl+Win+Left/Right for this, which is not comfortable
# during normal mouse-driven work. mHD can put that action under your thumb.
[[binding]]
trigger = "mouseButton4"
action = "replace_key"
keys = "ctrl+win+left"

[[binding]]
trigger = "mouseButton5"
action = "replace_key"
keys = "ctrl+win+right"

# Brightness up / down through DDC/CI.
# Useful for external monitors where Windows brightness controls do not work.
[[binding]]
trigger = "ctrl+alt+numpad_add"
action = "brightness_up"
value = "5"

[[binding]]
trigger = "ctrl+alt+numpad_subtract"
action = "brightness_down"
value = "5"

# Open the compact volume mixer instead of the Windows sound settings.
[[binding]]
trigger = "ctrl+alt+numpad_star"
action = "show_volume_mixer"

# Suspend the focused app when it loses focus.
# Useful for heavy games/tools that should stop consuming resources in background.
[[binding]]
trigger = "ctrl+alt+f9"
```

```
action = "toggle_suspend_on_blur"

# Throttle the focused app when it loses focus.
# Less aggressive than suspend: the process keeps running, but with reduced CPU
pressure.
[[binding]]
trigger = "ctrl+alt+f10"
action = "toggle_throttle_on_blur"

# Launch Windows Terminal from a shortcut.
[[binding]]
trigger = "ctrl+alt+t"
action = "run_ps"
command = "Start-Process wt"
```

Themes

mHD loads JSON colour themes from:

```
%USERPROFILE%\.config\mhd\themes
```

Set the active theme in `config.toml`:

```
theme = "night_glass"
```

The file must exist at:

```
%USERPROFILE%\.config\mhd\themes\night_glass.json
```

Supported keys:

Key	Used for
`background`	OSD, About, editor, mixer background
`surface`	Edit control background
`border`	Separator lines
`text`	Primary text
`text.muted`	Labels, version, hints, status, secondary text
`element.active`	Accent / progress bar fill

Key	Used for
`element.selected`	Selection highlight
`element.hover`	Hover state

Missing keys fall back to the built-in default theme.

On first run, mHD also creates bundled theme files when they are missing:

- `carbon`
- `paper`
- `night_glass`
- `day_glass`
- `ember`

Project Structure

```
mhd/
├── Cargo.toml
├── llm-proxy/
│   ├── Cargo.toml
│   └── src/
│       ├── main.rs
│       ├── config.rs
│       ├── handlers.rs
│       ├── state.rs
│       └── providers/
│           ├── mod.rs
│           ├── anthropic.rs
│           └── upstream.rs
└── mhd-daemon/
    ├── Cargo.toml
    └── src/
        ├── main.rs
        ├── app.rs
        ├── core/
        │   ├── action.rs
        │   ├── hook.rs
        │   ├── llm_proxy.rs
        │   ├── native_theme.rs
        │   ├── platform.rs
        │   ├── trigger.rs
        │   └── worker.rs
        ├── config/
        │   ├── mod.rs
        │   ├── raw.rs
        │   └── path.rs
        ├── overlays/
        │   ├── about.rs
        │   ├── autostart.rs
```

```
│   ├── cpu_plan.rs
│   ├── draw.rs
│   ├── llm_models.rs
│   ├── monitor.rs
│   ├── note.rs
│   ├── pomodoro.rs
│   ├── power.rs
│   ├── suspend.rs
│   ├── throttle.rs
│   ├── topmost.rs
│   ├── tray.rs
│   └── volume.rs
├── osd/
│   ├── mod.rs
│   └── painter.rs
├── renderer.rs
└── win32/
    ├── mod.rs
    └── text_host.rs
```

Optional Developer Build

The normal release build does not include `blackbox`.

`blackbox` is left in the source tree as an optional compile-time module for personal developer builds. It is a small local SQLite-backed activity logger that can record daemon/session events, input activity categories, foreground app changes, quick notes, and a few custom tool events.

It writes to:

```
%USERPROFILE%\.config\mhd\blackbox\blackbox.db
```

Build it explicitly when you want that local diagnostic/history layer:

```
cargo build --release --features blackbox
```

It is intentionally opt-in and should be treated as a local build option, not as part of the normal product.